

PostgreSQL & SQLModel

Web Design and Programming

M.Sc. Le Duy Khanh

Faculty of Data Science and Artificial Intelligence

Session Outline

1. Relational Databases
2. Connecting to PostgreSQL
3. ORM & SQLModel
4. Defining Models
5. Engine, Session & Tables
6. CRUD with SQLModel
7. Migrations with Alembic
8. Strategy for DS & AI Students
9. Worked Example & Labs
10. Review & Summary

Learning Objectives

By the end of this session, you will be able to:

1. Explain **table, row, column, primary key, foreign key** and one-to-many / many-to-many.
2. Build a $\mathrm{DATABASE_URL}$ for PostgreSQL/SQLite and read it from the environment.
3. Explain what an **ORM** is and what **SQLModel** combines.
4. Define tables with $\mathrm{table{=}True}$, $\mathrm{Field}(\ldots)$, $\mathrm{Relationship}()$.
5. Create an **engine**, run $\mathrm{create_all}$, use a $\mathrm{Session}$ per request.
6. Implement full **CRUD** inside path operations.
7. Explain why schema changes need **migrations** (Alembic).

Where We Are

- Week 7: full CRUD endpoints – but data lived in a Python list and **vanished on restart**.
- Week 8: the `get_session yield`-dependency.
- This week: a **persistent home** for data – a relational database, accessed via **SQLModel**.
- One set of Python classes = **database tables** *and* **API schemas**.

Relational Databases

Why a Database?

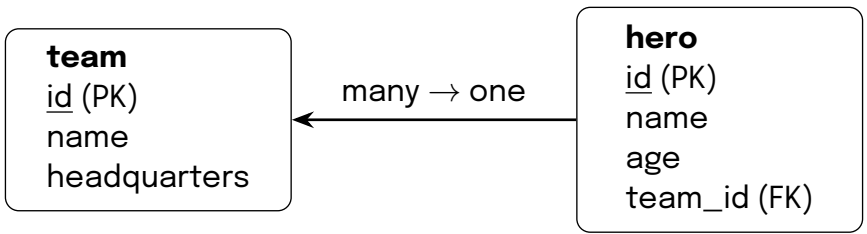
- “Database” can mean: the server *type* (PostgreSQL, SQLite), a *running instance*, or a *collection of tables*.
- A database solves problems you’d otherwise code yourself:
  - **Multiple access** – many requests reading/writing at once.
  - **Indexing** – fast lookup.
  - **Data consistency** – constraints, transactions.
- Relational databases use **SQL**; NoSQL (Redis, MongoDB) relax the fixed schema.

Tables, Rows, Columns, Keys

Concept	Meaning	DataFrame analogy
Table	Named collection of records	A DataFrame
Row (record)	One entity, e.g. one hero	One row
Column	Named, **typed** property	A column with a dtype
Primary key	**Uniquely identifies each row**	A unique index
Foreign key	References another table's PK	A join key

- PostgreSQL types: INTEGER, SERIAL (auto-increment), VARCHAR/TEXT, BOOLEAN, TIMESTAMP, NUMERIC (exact decimals).

Relationships



- **One-to-many:** one team has many heroes $\Rightarrow$ FK $\mathrm{hero.team_id} \rightarrow \mathrm{team.id}$ on the **"many" side**.
- **Many-to-many:** students $\leftrightarrow$ courses $\Rightarrow$ a **link (junction) table** $\mathrm{enrollment(student_id,\ course_id)}$ with two FKs.

Connecting to PostgreSQL

The Connection String

```
postgresql://user:password@host:5432/dbname
sqlite:///./app.db
```

- URL = dialect/driver + credentials + host + port (5432 = PostgreSQL default) + database name.
- **SQLite**: no separate server – one file, library runs in your process. Great for development/tests.
- SQLModel hides dialect differences $\Rightarrow$ switching SQLite $\rightarrow$ PostgreSQL = **change the URL only**.

Never Hardcode It: DATABASE_URL

```python
import os
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./app.db")
```

```
export DATABASE_URL="postgresql://app:secret@localhost:5432/appdb"
pip install "fastapi[standard]" sqlmodel "psycopg[binary]"
```

- Credentials in code end up in Git – read them from an **environment variable** (Twelve-Factor App).
- Same trick points tests at a separate test database.

Running PostgreSQL Locally

```
# macOS (Homebrew)
brew install postgresql@16
brew services start postgresql@16
# Ubuntu/Debian
sudo apt install postgresql && sudo service postgresql start
# Windows: installer from postgresql.org (includes psql, pgAdmin)
```

```
psql postgres            # Linux: sudo -u postgres psql
CREATE USER app WITH PASSWORD 'secret';
CREATE DATABASE appdb OWNER app;
\q
psql "postgresql://app:secret@localhost:5432/appdb"   # test it
```

- The server runs as a background service on port 5432; psql is the command-line client.

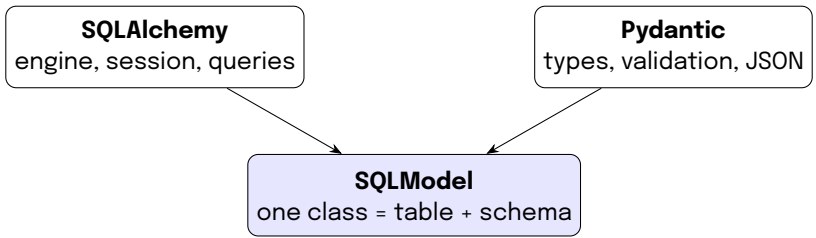
ORM & SQLModel

Levels of Database Access

Level	You write	Example
DB-API (raw SQL)	SQL strings + params	curs.execute("select ...", params)
SQLAlchemy Core	Functions generating SQL	select(hero_table).where(...)
ORM	Classes & objects	session.get(Hero, 1)

- **ORM (Object-Relational Mapping):** rows ↔ objects, tables ↔ classes.
- Work with Python objects/methods instead of hand-written SQL strings.
- Bonus: type checking, autocompletion, portability, **parameterized queries** (no SQL injection).
- Cost: one more abstraction – still understand the SQL it generates.

SQLModel = SQLAlchemy + Pydantic



- Written by the author of FastAPI.
- No duplicated field lists between “ORM model” and “API schema”.

Defining Models

Defining a Table

```python
from sqlmodel import SQLModel, Field, Relationship

class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True, unique=True)
    headquarters: str
    heroes: list["Hero"] = Relationship(back_populates="team")

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = None
    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")
```

Reading the Model

- table=True ⇒ a **real database table**. Without it: only a data schema (request/response, Week 7).
- Field(...): primary_key, index, unique, foreign_key="team.id", defaults.
- id = None before insert; the database generates it (SERIAL).
- Table name = lowercase class name (Hero → hero).
- Relationship(): **bidirectional** link – hero.team, team.heroes.
  - back_populates names the attribute on the other side.
  - Not a column – only team_id is stored.

One Model, Two Roles: API Schemas

```python
class HeroBase(SQLModel):                    # shared fields - not a table
    name: str = Field(index=True)
    age: int | None = None
    team_id: int | None = Field(default=None, foreign_key="team.id")

class Hero(HeroBase, table=True):            # the real table
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str                         # stored, never returned

class HeroCreate(HeroBase):                  # POST body
    secret_name: str

class HeroPublic(HeroBase):                  # response_model
    id: int

class HeroUpdate(SQLModel):                  # PATCH body: all optional
    name: str | None = None
    age: int | None = None
    team_id: int | None = None
```

Many-to-Many: Link Table

```python
class HeroMissionLink(SQLModel, table=True):
    hero_id: int | None = Field(default=None, foreign_key="hero.id",
                                primary_key=True)
    mission_id: int | None = Field(default=None, foreign_key="mission.id",
                                   primary_key=True)

# in Hero:
missions: list["Mission"] = Relationship(back_populates="heroes",
                                         link_model=HeroMissionLink)
```

- Composite primary key (hero_id, mission_id); two foreign keys.
- link_model= tells Relationship() to go through the link table.

Engine, Session & Tables

Engine & Creating Tables

```python
from sqlmodel import create_engine, SQLModel

connect_args = {"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}
engine = create_engine(DATABASE_URL, echo=True, connect_args=connect_args)

@asynccontextmanager
async def lifespan(app: FastAPI):
    SQLModel.metadata.create_all(engine)   # create missing tables
    yield

app = FastAPI(lifespan=lifespan)
```

- **One engine per app**: connection pool + dialect. `echo=True` prints generated SQL.
- `create_all` creates tables for imported `table=True` models, **skips existing ones**.
- OK for development; production uses **migrations**.

Session = Unit of Work per Request

```python
def get_session():
    with Session(engine) as session:
        yield session           # request runs here; closes afterwards

SessionDep = Annotated[Session, Depends(get_session)]
```



- Collects changes; sends them as one **transaction** on commit() (or rollback()).

CRUD with SQLModel

CRUD at a Glance

Op	SQLModel	SQL
Create	add(obj); commit(); refresh(obj)	INSERT
Read one	session.get(Hero, id)	SELECT ... WHERE id=?
Read many	session.exec(select(Hero).where(...)).all()	SELECT ... WHERE
Update	load → set attrs → add → commit	UPDATE
Delete	session.delete(obj); commit()	DELETE

Create & Read

```python
hero = Hero(name="Tony", age=45, secret_name="Iron Man", team_id=1)
session.add(hero)        # staged only - nothing written yet
session.commit()         # INSERT executed, transaction saved
session.refresh(hero)    # reload -> hero.id now filled in

hero   = session.get(Hero, 1)                                  # by PK, or None
heroes = session.exec(select(Hero)).all()                      # list of Hero
adults = session.exec(select(Hero).where(Hero.age >= 18)).all()
tony   = session.exec(select(Hero).where(Hero.name == "Tony")).first()
page   = session.exec(select(Hero).order_by(Hero.id).offset(20).limit(10)).all()
```

- commit() **persists**; add() alone writes nothing.
- exec(select(Model)).all() returns a **list of model objects**.

SQL vs SQLModel vs pandas

SQL	SQLModel	pandas
WHERE name = 'X'	select(Hero).where(Hero.name == "X")	df[df.name == "X"]
ORDER BY age DESC	.order_by(Hero.age.desc())	df.sort_values("age", ascending=False)
LIMIT 10 OFFSET 20	.offset(20).limit(10)	df.iloc[20:30]

- The ORM is a convenient wrapper – turn on `echo=True` to see the SQL.
- Filter, sort, paginate **in the database**, not by loading every row into Python.

Update, Delete & Errors

```
hero = session.get(Hero, 1)
hero.age = 46
session.add(hero); session.commit(); session.refresh(hero)   # update

session.delete(hero); session.commit()                        # delete
```

- **Missing:** get returns None $\Rightarrow$ HTTPException(404), not a 500 crash.
- **Duplicate:** violating unique=True raises IntegrityError $\Rightarrow$ rollback(), return 409 Conflict.

Migrations with Alembic

Why Migrations?

- You add $\mathrm{Hero.power}$ and restart $\Rightarrow$ $\mathrm{create_all}$ does **nothing**: the table already exists.
- Drop & recreate? Destroys data.
- A **migration** = versioned script that changes the schema step by step.
- **Alembic** (from the SQLAlchemy author) manages them:
  - Each revision has $\mathrm{upgrade}()$ and $\mathrm{downgrade}()$ $\Rightarrow$ **history + rollback**.
  - Kept in Git, applied identically on every machine.

Alembic Workflow

```
pip install alembic
alembic init migrations          # migrations/ + alembic.ini
# migrations/env.py: target_metadata = SQLModel.metadata
alembic revision --autogenerate -m "add hero.power"   # diff models vs DB
alembic upgrade head             # apply pending migrations
alembic downgrade -1             # roll back the last one
alembic history                  # list versions
```

- **Always review** autogenerated scripts – a rename looks like drop + add.

Strategy for DS & AI Students

Learning Strategy

- **You know SQL and DataFrames:** $\text{select}().\text{where}() \approx \text{SELECT } \ldots \text{ WHERE} \approx \text{df}[\text{df.name} == \text{"X"}]$.
- **One model, two roles:** DB table + API schema – less duplication.
- **Session = unit of work:** tied to one request – that's why get_session uses yield.
- **Seeding $\approx$ loading a DataFrame:** write a seed script so upcoming sessions have real data.

Worked Example & Labs

Worked Example: Create & List

```python
@app.post("/heroes", response_model=HeroPublic, status_code=201)
def create_hero(hero_in: HeroCreate, session: SessionDep):
    hero = Hero.model_validate(hero_in)
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes", response_model=list[HeroPublic])
def list_heroes(session: SessionDep, offset: int = 0,
                limit: int = Query(default=10, le=100)):
    return session.exec(
        select(Hero).order_by(Hero.id).offset(offset).limit(limit)).all()
```

- response_model=HeroPublic hides secret_name.

Worked Example: Read, Update, Delete

```python
@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero_in: HeroUpdate, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero.sqlmodel_update(hero_in.model_dump(exclude_unset=True))
    session.add(hero); session.commit(); session.refresh(hero)
    return hero

@app.delete("/heroes/{hero_id}", status_code=204)
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero); session.commit()
```

Worked Example: Seed Script

```python
def seed():
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        avengers = Team(name="Avengers", headquarters="New York")
        xmen = Team(name="X-Men", headquarters="Westchester")
        session.add_all([
            Hero(name="Tony", age=45, secret_name="Iron Man", team=avengers),
            Hero(name="Natasha", age=35, secret_name="Black Widow", team=avengers),
            Hero(name="Logan", age=150, secret_name="Wolverine", team=xmen),
        ])
        session.commit()
```

```
python -m app.seed
fastapi dev app/main.py      # then try every endpoint at /docs
```

- team=avengers: SQLModel inserts the team first and fills team_id.

Lab 1: Models & Tables

Hands-on practice

- Define Team and Hero (one-to-many) with primary_key, index, foreign_key.
- Create the engine; call SQLModel.metadata.create_all(engine).
- Verify: sqlite3 app.db ".schema" or psql -c "\dt"; read the echo=True output.

Lab 2: CRUD Endpoints

Hands-on practice

- Move your Week 7 in-memory CRUD onto the real DB with SessionDep.
- POST (201), GET list with offset/limit, GET /{id} (404), PATCH, DELETE (204).
- Use response_model to return only the public schema.
- Restart the server – confirm the data **survives**.

Lab 3: Project Integration

Hands-on practice

- As a group, finalize your project's schema: draw tables, columns, keys, relationships.
- Write the matching SQLModel models ($\text{table}=\text{True}$ + $\text{Create}/\text{Public}/\text{Update}$).
- Write seed.py with realistic sample data for the frontend/dashboard.

Review & Summary

Quick MCQ Review

1. What is a primary key for? **Uniquely identifies each row in a table**
2. Main benefit of an ORM? **Work with Python objects/methods instead of SQL strings**
3. What does $\texttt{table=True}$ change? **Class becomes a real DB table, not just a schema**
4. After $\texttt{session.add(...)}$, what saves to the DB? $\texttt{session.commit()}$
5. $\texttt{session.exec(select(Model)).all()}$ returns? **A list of matching model objects**
6. What is Alembic for? **Versioned schema migrations – upgrade/downgrade**
7. New field, restart – why no new column? $\texttt{create_all}$ **never alters existing tables**
8. FK in Team-Hero one-to-many goes where? **On the "many" side:** $\texttt{hero.team_id}$

Self-Check Checklist

- ☐ Can explain PK, FK, one-to-many, many-to-many
- ☐ Can build a DATABASE_URL and read it from the environment
- ☐ Can explain ORM and what SQLModel combines
- ☐ Can define models with table=True, Field, Relationship
- ☐ Can set up engine, create_all, and SessionDep
- ☐ Can write all five CRUD endpoints with 404 handling
- ☐ Know when and why to use Alembic migrations

Summary and Next Session

- **Relational DB**: typed tables; **PK** identifies rows, **FK** links tables; many-to-many via a link table.
- `DATABASE_URL` from the environment; SQLite for dev, PostgreSQL for prod – same code.
- **ORM** maps rows to objects; **SQLModel** = SQLAlchemy + Pydantic: one class = table + schema.
- `Field(...)` for keys/indexes; `Relationship()` for two-way navigation.
- One **engine** per app; `create_all` creates missing tables; one `Session` per request.
- **CRUD**: `add`+`commit`+`refresh`; `get`/`exec(select())`; modify+`commit`; `delete`+`commit`.
- Schema changes need **migrations**: Alembic `revision --autogenerate`, `upgrade head`, `downgrade`.
- Next (Week 10): RESTful design, auto-docs, API tests with `pytest` + `TestClient`.

Q&A

Questions & Answers